LABORATORY 1

DESIGNING AND PROGRAMMING A DISCRETE OBJECT CLASS

Originally designed by: dr inż. Łukasz Maliński

Programming issues covered:
Classes and encapsulation.
• Abstract class.
• Inheritance and dynamic object polymorphism.
• Random number generators – <random> library.
• Programming rules: DRY (Don’t Repeat Yourself), YAGNI (You Aren't Gonna Need It) and KISS.
• Elementary code refactoring (separating a method/class).
• Object serialization and deserialization

Mathematical modelling topics covered:
• ARX model simulation.
• Generating random numbers with a normal distribution.

Skills to master:
• Creating data abstractions to model a given problem.
• Using inheritance and polymorphism.
• Implementation of an algorithm simulating the operation of a simple model of a dynamic object.
• Basics of code testing and refactoring.
• Writing classes that test other classes, class friendships.
• Implementing an object serialization and deserialization algorithm

Command designations:
■ Commands that are mandatory and necessary for the implementation of the entire project being implemented during all class topics.
● Optional commands - allow you to get better grades.

Getting Started :

a. Run Code::Blocks or other programming environment (IDE) of your choice that supports C++.
b. Create a new empty project with a name „PfcsLab” or similar. In Code::Blocks, this can be done by selecting: **File** >> **New** >> **Project**, then **Empty project**, with **Release** configuration. You may place your project folder in the Documents, or Desktop, or another location of your choice.
   Note: You are going to use the same project over many lab exercises, because each week we are going to expand the same project, adding more functionalities and more classes.
c. Create a new empty file called **main.cpp** in a new subfolder **src** which will be placed in your project folder. (This src folder will be used for storing all cpp files that you create

during all the laboratory exercises.) In Code::Blocks, you can do it by selecting: File >> New >> File..., then C/C++ source or Empty file.

d. In the main.cpp file, write a simple program of „Hello world" type: include the necessary header(s); write the main() function; inside it, print some simple text to the console (std::cout). Build and run the program.

e. Once this works, you may move to the main topic of today, which is programming a class that simulates a ObjectSISO in discrete time.

Tasks to be done:

Discrete Object Class:
1.Define an abstract class, ObjectSISO, containing only one purely virtual (no implementation) method (member function), named simulate. This method should accept a single sample of the input signal u as an argument and return a single sample of the output signal y. Place this class in a separate header file.

■ **Tips:**
This class will be the base for most of the other classes defined in subsequent classes, so create it to provide all the necessary polymorphic properties. Remember that for polymorphism to work correctly, the class's destructor should be virtual.

Do not define any constructors, as this class has no internal state that needs to be initialized. Since this class does not contain any executable code, there is no need to create a CPP file for it. Note that the signals in our simulation problem take real values.

2. Define the ModelARX class as a derivative of ObjectSISO. The class should contain fields (data components) enabling the simulation of the ARX model (see the lecture). In particular, it should contain:
- two coefficient vectors for the polynomials A (denominator) and B (numerator),
- a variable defining the order of the transport delay (minimum 1),
- the standard deviation of the noise or an object forming the normal distribution.
- memory (queue) for input signals,
- memory (queue) for output signals,
- memory (queue) for the transport delay.

Tips:
- The polynomial's coefficients take real values
- The order of the transport delay is always positive.
- The standard deviation is a non-negative real number.
- Special container classes (queues) in C++ are designed to store signal memory (buffering).

3. Define the ModelARX class interface (public methods) according to the following guidelines. The class should provide:
- Setters for coefficient vectors for both polynomials.

Tips:

- Note that polynomial setters should also define the sizes of the signal buffers. There should be as many signal samples stored as there are coefficients.
- Also, remember that the polynomial A has a zero coefficient (corresponding to sample y(i)), always equal to 1.0. It can therefore be omitted when storing.
- Absolutely do not define public setters for buffer sizes alone, as this information is strictly dependent on the number of coefficients. They cannot be set independently!
- A transport delay setter, which prevents the entry of a non-positive value and sets the size of the corresponding buffer.
- A standard deviation setter for the interference signal, which prevents the entry of a negative value.

Hints:
Value constraints must be implemented manually in the setter body (as value-checking code). Simply defining the delay setter argument as unsigned does not solve the problem, as a negative value can still be entered. The problem is that a negative value will be binary interpreted as a very large positive value. Entering an invalid value should result in a default value of 1 for the delay and 0.0 for the standard deviation.
- an overloaded virtual method, simulate. Do not define its implementation yet; return 0.0 to avoid compilation errors.
- an argument constructor allowing the entry of the coefficients of both polynomials, the order of the transport delay, and the standard deviation of the interference signal. For the last two arguments, define default values of 1 and 0.0, respectively.

Tips:

- Defining an argument constructor automatically excludes the default constructor, but it's good programming practice to explicitly exclude it with the delete specifier. An argument constructor should definitely use defined setters to avoid unnecessary code duplication (DRY rule). Don't set field values directly, or in the constructor's initialization list, as this not only circumvents constraints but also bypasses the execution of important code responsible for adjusting buffer sizes.
- Stick with the YAGNI rule to avoid defining anything unnecessary. Therefore, there's no need to define copy and move constructors, let alone their corresponding assignment operators, because at this stage the class doesn't have any external dependencies that require redefining their default versions (created automatically by the compiler). There's also no need to define getters for fields. If such a need arises in the future, they can always be added.

4. Place the ModelARX class in a separate translation unit (a pair of h and cpp files).

Tips:
Be sure to remember that the header file and the source file should have the same name.

5. Write a temporary version of the private method to simulate a single sample of the interference signal. This version should only return 0.0.

Tips:

At this stage, we don't want to bother with noise simulation, but this method is needed in the simulation code. Returning 0.0 will not interfere with work on the simulate method.

6. Define an implementation of the virtual simulate method so that it correctly simulates the operation of the ARX model. The model output is:

$$y_i = (b_0 u_{i-k} + b_1 u_{i-k-1} + \cdots b_{dB} u_{i-k-dB}) - (a_1 y_{i-1} + a_2 y_{i-2} + \cdots + a_{dA} y_{i-dA}) + e_i$$

$b_0, b_1, \ldots, b_{dB}$ are the coefficients of polynomial B,
$a_1, \ldots, a_{dA}$ are the coefficients of polynomial A,
k is the order of transport delay,
u_i are the samples of the input signal,
y_i are the samples of the output signal
$e_i$is the random value of the disturbance (taken from the just defined private method for noise simulation).

Simulation Testing:
■ Run the ModelARX cash register unit tests provided in the "lab1.cpp" file to verify the correctness of the simulation. If the tests fail, correct the simulation code until successful. Assume that the verification in the tests is working correctly. If the test code does not conform to the class interface, adjust it.

Tips:
Note that if the tests fail, they print the expected and actual results. The display precision and numerical tolerance of the tests can be changed using the PREC and TOL constants in the auxiliary function code.
The tests verify signal values with a small tolerance. This is important when comparing real numbers, as minor numerical errors are possible, which do not necessarily indicate implementation errors. The test values were also written with limited precision! These tests assume zero noise variance, as otherwise the output signal values are unpredictable and impossible to test with such simple precision.
■ Only after the simulate method passes all tests with "OK!", implement a private method to simulate a single sample of the disturbance signal. The disturbance should have a zero expected value and a configurable standard deviation.

Tips:
This allows for freedom of implementation. You can use any generator, e.g., mt19937, random_device, default_random_engine, etc. It is important to pass the generator output through an appropriate normal distribution forming object with specified statistical parameters. The class can store just the standard deviation, the entire forming object, or use std::bind() and std::function.

● After analyzing the code generated after completing the basic task, you may notice some inelegant repetitions in the simulate function code (related to the red brackets) and in the ModelARX class itself. To improve this, you can:
- Separate the code fragments that simulate the selected polynomial into a private method in the ModelARX class and use it in the simulate method.

- Separate all the functionality for processing polynomials and the data required for this into separate instances of the new Polynomial class (so-called composition).

Decide which of them you think better fits the KISS principle and refactor the code accordingly (reorganize the code without changing its functionality). The first solution requires fewer changes.

but requires a bit more programming dexterity. The second is more time-consuming and interferes more with the class code, but is slightly easier syntactically.

NOTE: After refactoring, be sure to check whether the class passes the tests! It's very important not to change the ModelARX class interface, as it's crucial in object-oriented programming that the use of a class be as little dependent on its internal implementation as possible (the encapsulation rule). Refactoring changes within a class shouldn't modify the code using it. Other classes won't care whether the ModelARX class uses separate instances of the Polynomial class, private methods, or simulates everything in a single method.

● Move the tests from the lab1.cpp file to a purely static class, Testy_ModelARX, which contains only static members. Make this entire class friends with the ModelARX class so that it has access to all the functionality of the class being tested (even those not accessible through its interface). Place the tests and their helper functions in this class as static private methods, and write a static public method that calls them all. Place the class in the same translation unit as the ModelARX class. There's no point in keeping them separate.

Note: Generally, class friendship is rarely justified, but in this case, we don't need to be in any doubt. Code testing a functionality should be privileged to gain a complete and accurate picture of the situation. It also often needs access to something intentionally kept private, such as the signal memory buffers in our example.

Class Serialization and Deserialization:
■ In the ModelARX class, implement a utility method to serialize an object instance. Serialization is the process of converting an instance into a sequence of bytes or characters so that it can be saved to files, transmitted over the network, etc. Choose your own serialization method. You can:
- (●) develop your own ASCII or binary format,
- use ready-made solutions such as JSON or boost::archive.

Tips:
When using ready-made solutions, you need to adapt the class to work with them. It's better to study several examples available in the tool's documentation and modify your class based on them. When designing your own algorithm, note that container classes store data in the heap, which requires careful extraction. If you have any pointers in your class, remember that the addresses stored in them are no longer valid after the program is restarted (do not save any addresses in serialization).
■ In the ModelARX class, add a conversion constructor that deserializes the instance from the format selected/developed in the previous command. The constructor should recreate the entire object state.

Tips: The role of this constructor is to create an object with the state read from the serialized data. Define it, even if you are using ready-made tools that provide other solutions for object reconstruction, define this construct. We want each class to be responsible for its own serialization and deserialization, so that it can be used later to serialize more complex data structures.

● Add a separate unit test that verifies whether the serialization process is properly reversible. To do this, serialize a single instance to a file on your hard drive, then read it and create a new instance using the deserialization constructor. The test result should answer the question whether the original and reconstructed instances are identical.

Tips: Writing your own == operator for our ModelARX class can be very helpful. Note that because the class contains vectors, the std::memcmp binary comparison cannot be used here.